PROJECT FEATURE GUIDE

Silt - AI Data Analyst

Every feature: what it does, what it is built with, and how it is shown in the app.

Product	Silt (the in-app name). The repository and marketing site call it DataSage.
One-line summary	Upload a CSV, ask a business question in plain English, and get a dashboard whose every number is traced back to the code, data and verification that produced it.
Main stack	Python, FastAPI, LangGraph, Streamlit, SQLAlchemy, PostgreSQL / SQLite, pandas, Plotly, Docker
Date	16 October 2026

1. Overview

Silt is an AI data analyst for teams. A user uploads a CSV file and asks a question such as *"Why did revenue drop in March?"*. A pipeline of AI agents plans the analysis, writes and runs Python code in a locked-down sandbox, has an independent **Critic** re-check the results, and publishes a dashboard of KPIs, charts and a written summary. Every number on that dashboard can be clicked to see the exact code, data rows and verification behind it.

Where the answer does not need a model, Silt answers **deterministically** (plain code, no AI): simple totals, comparisons and statistical tests, root-cause breakdowns and what-if scenarios all have model-free paths. This keeps answers cheaper, faster and reproducible.

Architecture

Layer	What runs there	Technology
Marketing site	Public landing page that links into the app	Next.js 16, React 19, Tailwind CSS v4
Web app (UI)	17 pages: datasets, analyses, dashboards, audit trail, teams, settings ...	Streamlit 1.62, Plotly, custom CSS design system (style/theme.py)
API	REST endpoints for auth, data, analyses, dashboards, teams	FastAPI, Pydantic, Uvicorn, JWT (PyJWT), bcrypt
AI pipeline	Router, Triage, Planner, Executor, Critic, Dashboard agents	LangGraph state machine; Google Gemini with Llama failover
Analysis engines	Statistics, metric formulas, root cause, scenarios, data quality	pandas, NumPy, SciPy (no model involved)
Code sandbox	Runs the Python the agents write, isolated from the app	Docker container: no network, non-root, CPU / memory / time limits
Database	Users, teams, datasets, questions, dashboards, audit trail, jobs ...	SQLAlchemy 2.0 + Alembic; PostgreSQL (Neon) with pgvector in production, SQLite locally
Background work	Durable job queue and scheduled re-runs	Database-backed job queue, APScheduler
Deployment	One container serving API + UI	Docker, start.sh, Render blueprint (render.yaml)

How a question becomes a dashboard

Step	What happens
1. Upload	The CSV is streamed to disk (size-capped), then profiled: column types, row count, missing values, basic statistics.
2. Ask	POST /api/questions stores the question and puts an analysis job on the queue. An Idempotency-Key header stops double submissions.
3. Route	The Router picks the cheapest path that can answer properly (see table below) and records why.
4. Plan and execute	The Planner breaks the question into steps; the Executor writes Python for each step and runs it in the sandbox, retrying with the error message if it fails.
5. Verify	The Critic independently recomputes the key numbers in its own sandbox and accepts or rejects each one.
6. Publish	The Dashboard agent builds KPIs, charts and a narrative, and writes audit-trail rows linking each element to its code, data and Critic review.

Route	Used when	Model calls
clarify	The question can be read more than one way; the user is asked to pick	-
fast	A simple aggregation (totals, averages, counts)	None
comparison	"Is there a difference between A and B?" - answered by the statistics engine	1
statistical	Questions needing a formal statistical method	Few
root_cause	"Why did X change?" - deterministic decomposition	None
hypothesis	Open-ended "why" questions over time data - model proposes, code tests	Bounded
standard	Everything else - full Planner / Executor / Critic run	Several

2. Features

Each card names the feature and where to find it in the app's sidebar, then explains what it does, what it is built with, and how it appears on screen.

A. Accounts and teams

Sign up and log in

Login screen (before any page)

WHAT IT DOES	Creates an account and signs the user in. New accounts automatically claim any team invites already sent to that email address.
BUILT WITH	FastAPI `/api/auth` routes, bcrypt password hashing, JWT tokens (HS256, 7-day expiry), in-memory rate limiting on login.
HOW IT IS SHOWN	A full-screen login / sign-up form. Once signed in, the user's name and a log-out button appear at the bottom of the sidebar.

Workspaces: communities and teams

Sidebar - Workspaces

WHAT IT DOES	Users create communities and teams inside them. All data is private to a team: every request carries the active team and the server checks membership.
BUILT WITH	FastAPI workspaces router, `X-Team-Id` header + membership check, SQLAlchemy models (communities, teams, team_members).
HOW IT IS SHOWN	Lists of communities and teams with a Select button; a team switcher (two drop-downs) at the top of the sidebar. Pages that need a team show a "You're not on a team yet" message with a button to Workspaces.

Team invites by email

Workspaces - Invite one / Invite many

WHAT IT DOES	Owners and admins invite people by email, singly or in bulk. A real email is sent, invites can be accepted, and members can be removed. Malformed addresses are rejected.
BUILT WITH	SMTP email (Gmail by default) in `email_sender.py`; pending membership rows with `invited_email`.
HOW IT IS SHOWN	Two tabs, "Invite one" and "Invite many at once"; a members table with roles; a notice saying whether the email was actually delivered; pending invites with an Accept button.

Team overview

Sidebar - Team overview

WHAT IT DOES	A timeline of every analysis the team has run.
BUILT WITH	Streamlit page reading `GET /api/questions`.
HOW IT IS SHOWN	A chronological list, newest first, each with a status badge (Verified, Running, Rejected).

Settings

Sidebar - Settings

WHAT IT DOES	Shows the signed-in account, the active community / team and role, and a read-only view of the live backend configuration (models, job workers, version).
BUILT WITH	Streamlit page reading `/api/auth/me` and `/api/health`.
HOW IT IS SHOWN	Account and team cards with a role badge, plus a configuration panel.

B. Data

Dataset upload and profiling

Sidebar - Datasets - Upload Data

WHAT IT DOES	Uploads a CSV (up to 200 MB by default) and profiles it automatically: column types, row and column counts, missing values and basic statistics.
BUILT WITH	FastAPI multipart upload streamed to disk, pandas profiling in `profiling.py`, stored as JSON / JSONB.
HOW IT IS SHOWN	An Upload Data button opens a drag-and-drop box and a "Profile this dataset" button; the dataset table lists each file with rows, columns and upload time; selecting one shows its column profile.

Dataset versions (Replace data)

Datasets - Replace data

WHAT IT DOES	Uploads a new version of an existing dataset so analyses can be re-run on fresh data while keeping the history. Each version is fingerprinted for reproducibility.
BUILT WITH	`dataset_versions` table, `provenance.py` fingerprints.
HOW IT IS SHOWN	A "Replace data - currently vN" expander on the dataset; the version number is shown on the dataset row.

Auto-insights

Datasets (selected dataset)

WHAT IT DOES	Generates headline observations about a freshly uploaded dataset.
BUILT WITH	`insights.py` and the insights router.
HOW IT IS SHOWN	An insights panel on the dataset view, with a button to generate them.

Correlation explorer

Datasets (selected dataset)

WHAT IT DOES	Finds the strongest relationships between numeric columns. Purely deterministic, no AI.
BUILT WITH	pandas / NumPy in `correlations.py`.
HOW IT IS SHOWN	A ranked list of column pairs with their correlation strength.

Knowledge notes and lessons

Datasets - notes tabs

WHAT IT DOES	Short team-owned notes about a dataset (meaning of columns, known issues, lessons). The Planner agent is given these notes when it plans an analysis.
BUILT WITH	`knowledge_notes` table with optional embeddings; knowledge router (CRUD).
HOW IT IS SHOWN	Tabs for each kind of note, with an add box and delete buttons.

Data quality monitoring

Sidebar - Data quality

WHAT IT DOES	Runs quality checks on every dataset version, measures drift against a baseline version, and opens incidents when a threshold is crossed.
BUILT WITH	`data_quality.py`, quality router, dq_snapshots / dq_configs / dq_incidents tables.
HOW IT IS SHOWN	Quality results per version, a "Compare two versions" expander, a "Baseline and thresholds" settings expander, and an incident list with Acknowledge / Resolve actions.

C. Asking questions and analysis

Analyses: ask a question and watch it run

Sidebar - Analyses

WHAT IT DOES	Pick a dataset, type a question, and watch the agents plan, execute and verify it live. Runs are durable jobs that can be cancelled or retried.
BUILT WITH	LangGraph pipeline (Router, Triage, Planner, Executor, Critic, Dashboard); Gemini with automatic Llama failover; database-backed job queue (`jobs.py`); Docker sandbox.
HOW IT IS SHOWN	A question box and dataset picker; a live progress view showing each stage and step updating every few seconds; a "Run trace" expander with the chosen route, model calls, code runs and cost.

Smart routing and fast path

Automatic, inside Analyses

WHAT IT DOES	Chooses the cheapest correct path for each question. Simple aggregations are answered with no model call and no generated code; ambiguous questions ask the user to clarify.
BUILT WITH	`agents/router.py`, `fastpath.py`, per-route budgets in `runtime.py`.
HOW IT IS SHOWN	The chosen route and the reasons for it are shown in the run trace; clarification options appear as buttons.

Statistical reasoning and comparisons

Automatic, inside Analyses

WHAT IT DOES	Answers "is there a difference?" and similar questions with a formal statistical method chosen by rules, not by the model. Each result records why the method was chosen, which assumptions were checked and what the result does not show.
BUILT WITH	`stats/` package (method selection, tests, confidence intervals, robust regression), SciPy, NumPy; `comparison.py`.
HOW IT IS SHOWN	Results shown with estimate, interval and test, plus a "Technical details of the statistical methods" expander.

Copilot

Sidebar - Copilot

WHAT IT DOES	A multi-turn chat over one dataset: ask, add filters, drill down. Every answer is recomputed from the data; the conversation only records the decisions taken.
BUILT WITH	`copilot.py`, copilot router, copilot_sessions / copilot_turns tables.
HOW IT IS SHOWN	A chat interface with a message box ("Ask about this dataset, add a filter, or drill down") and a list of saved sessions.

Root-cause investigations

Sidebar - Root cause

WHAT IT DOES	Explains where a change in a metric came from: contribution by segment, price versus volume, tested hypotheses, and what could not be established. Works without a model; an optional hypothesis engine lets the model propose ideas that code then tests.
BUILT WITH	`investigator.py`, `hypothesis_engine.py`, `analysis_ops.py`, investigations router.
HOW IT IS SHOWN	A "New investigation" form; a report with segment contributions, accepted hypotheses, an expander of hypotheses rejected before testing, and an expander listing every analysis executed.

What-if scenarios

Sidebar - What-if

WHAT IT DOES	Scenario arithmetic on explicit inputs and assumptions (for example price +5%, volume -3%). Outputs are labelled as scenarios, never forecasts.
BUILT WITH	Deterministic models in `scenarios.py`, scenarios router.
HOW IT IS SHOWN	Two tabs, "Build a scenario" and "Saved scenarios"; expanders for the formulas used and for taking the baseline from a dataset; side-by-side comparison of scenarios.

Scheduled analyses and alerts

Sidebar - Scheduled

WHAT IT DOES	Saves a question to re-run on an interval. After each run the KPIs are compared with the last run, and the team is alerted only if a KPI moves past its threshold or the Critic cannot verify the result.
BUILT WITH	APScheduler (`scheduler.py`), `change_detection.py`, alerts router, email.
HOW IT IS SHOWN	A list of scheduled questions with pause / resume / delete; an alert inbox with Acknowledge and Resolve; an unread-alert count next to "Scheduled" in the sidebar.

D. Results and trust

Dashboards

Sidebar - Dashboards

WHAT IT DOES	The finished result of an analysis: KPI cards, charts and a written narrative, with an overall verdict - Verified, Verified with caveats, or Unverified.
BUILT WITH	Dashboard agent (`agents/dashboard.py`), Plotly figures stored as JSON, `verdict.py`.
HOW IT IS SHOWN	A row of KPI cards, chart cards and the narrative, each with a verification badge.

Chart Studio

Dashboards - Chart Studio bar

WHAT IT DOES	Re-draws any dashboard chart as one of 40 chart types and restyles it. Only the presentation changes; the verified numbers are never recomputed. Views are saved for the whole team.
BUILT WITH	`chart_studio.py` (pure functions, unit-tested), Plotly, `PUT/DELETE .../charts/{key}/view`.
HOW IT IS SHOWN	A toolbar above the dashboard to pick a chart type and style; the Inspect panel shows the exact Python code that drew the new chart and the formula it applies.

Click-to-inspect

Dashboards - any KPI or chart

WHAT IT DOES	For one dashboard element, shows the exact code, a plain-English formula and the data slice that produced it, including the original CSV line numbers.
BUILT WITH	Inspect router, audit_trail table, `csv_lines.py`.
HOW IT IS SHOWN	A pop-up dialog opened from the element, with code, formula and a table of the rows used.

Ask your dashboard

Dashboards - chat box at the bottom

WHAT IT DOES	Follow-up questions answered from one finished analysis.
BUILT WITH	`chat.py`, `POST /api/questions/{id}/chat`.
HOW IT IS SHOWN	A chat input: "Ask a follow-up about this analysis...".

Evidence, run record and reruns	Dashboards - evidence expander
WHAT IT DOES	Lists the evidence behind each claim, keeps a run manifest (dataset version, route, models), and lets the user rerun the analysis and compare the two runs.
BUILT WITH	`provenance.py`, `verification.py`, evidence router, evidence_records / run_manifests tables.
HOW IT IS SHOWN	Tabs "Evidence", "Run record" and "Rerun and compare", plus a "Download run record (JSON)" button.

Share links	Dashboards - share
WHAT IT DOES	Creates a read-only link to one verified dashboard, with an expiry (7 days by default), which can be revoked. Anyone with the link can view it without logging in.
BUILT WITH	Sharing router, share_links table with random tokens, public endpoint `/api/public/shared/{token}`.
HOW IT IS SHOWN	The link is created and revoked from the dashboard; the viewer sees a clean page (`/Shared?t=...`) with no sidebar and no login.

Audit trail	Sidebar - Audit trail
WHAT IT DOES	Traces every dashboard number back to the code, data and reasoning behind it, and to the Critic review that verified it.
BUILT WITH	audit_trail, execution_logs and critic_reviews tables; `storage/audit.py`.
HOW IT IS SHOWN	Four tabs: Overview, Code & Logs, Data Lineage and Verifications, with expanders for raw output and for the Critic's verification code and recomputed values.

Reports	Sidebar - Reports
WHAT IT DOES	Packages every completed analysis as a downloadable summary.
BUILT WITH	Streamlit page over the questions and dashboards endpoints.
HOW IT IS SHOWN	A list of completed analyses, each with a download button that saves a text report.

Overview and Home	Sidebar - Home / Overview
WHAT IT DOES	Home introduces the product and shows a live snapshot of the workspace; Overview shows counters, recent analyses, an insights chart and dashboards.
BUILT WITH	Streamlit, Plotly, custom stat-card components.
HOW IT IS SHOWN	Hero section and "how it works" on Home; stat cards, a recent-activity list and a chart on Overview.

E. Governance and quality of the AI

Semantic layer	Sidebar - Semantic layer
WHAT IT DOES	The team's approved business metric definitions (with versions, approval and deprecation), the words people use for them, and how they connect to datasets and columns. Metrics are computed by a small whitelisted formula language, with no AI involved.
BUILT WITH	`semantic.py`, `metric_engine.py`, semantic router and tables.
HOW IT IS SHOWN	Tabs: Metrics, New metric, Dimensions and entities, Knowledge graph, and Search.

Memory of past analyses (RAG)	Automatic
WHAT IT DOES	Each finished dashboard is embedded so the Planner can retrieve the most similar earlier analyses on the same team and dataset as context.
BUILT WITH	Gemini embeddings (768 dimensions), pgvector in PostgreSQL, `memory.py`.
HOW IT IS SHOWN	Not a separate screen; it improves plans in the background.

Evaluation suite	Sidebar - Evaluation
WHAT IT DOES	A regression test suite for the agents, run from the command line (`python -m app.evals run`). The page only reads the results.
BUILT WITH	`app/evals` runner with scripted cases and calibration.
HOW IT IS SHOWN	A list of runs; each case shows a pass / fail mark and expands to show details; runs can be compared.

F. Platform, security and deployment

Secure code sandbox	Behind every analysis
WHAT IT DOES	Runs the Python the agents write in isolation, so generated code cannot reach the app's data, network or secrets.
BUILT WITH	Docker container: no network, non-root user, 60 s time limit, 512 MB memory, 1 CPU, process and output limits (`sandbox/runner.py`).
HOW IT IS SHOWN	Not visible directly; its code and output appear in the Audit trail.

AI provider failover	Behind every model call
WHAT IT DOES	Uses Google Gemini by default and switches to a Llama model automatically if Gemini fails, saying which provider failed. Model calls are forced to return structured output.
BUILT WITH	`agents/llm_client.py` (google-genai), `llm_client_llama.py` (OpenAI-compatible client).
HOW IT IS SHOWN	The provider and cost per run appear in the run trace; models are listed on Settings.

Security hardening	Whole app
WHAT IT DOES	Escapes all user- and AI-written text before display (prevents stored XSS), hides internal errors, limits upload size, rate-limits login, and keeps every team's data separate.
BUILT WITH	`esc()` helper in the UI, FastAPI exception handlers, per-team checks on every query, structured logging.
HOW IT IS SHOWN	Invisible when working; users see clear error messages instead of tracebacks.

Testing and CI	Repository
WHAT IT DOES	Automated tests for the backend (auth, team isolation, statistics, routing, evals ...) and the frontend (Chart Studio, escaping), run in continuous integration. Dependency versions are pinned.
BUILT WITH	pytest, GitHub Actions.
HOW IT IS SHOWN	Not part of the app UI.

Deployment	Render / Docker
WHAT IT DOES	Ships the API and UI together in one container, with an external PostgreSQL (Neon) database.
BUILT WITH	Dockerfile, `start.sh` (starts the API, waits until healthy, then starts Streamlit), Render blueprint, Alembic migrations.
HOW IT IS SHOWN	Users simply open the app URL.

Marketing website	Separate site (web/)
WHAT IT DOES	A public landing page that explains the product and links to the app.
BUILT WITH	Next.js 16, React 19, Tailwind CSS v4, TypeScript; a printed "paper ledger" visual style.
HOW IT IS SHOWN	A static site with scroll-driven animations; its buttons open the Streamlit app.

3. Sidebar map

The order of pages in the app's sidebar, and the main feature on each.

Page	Main feature
Home	Product introduction and live workspace snapshot
Overview	Counters, recent analyses, insights chart
Datasets	Upload, profile, versions, insights, correlations, notes
Analyses	Ask a question and watch the agents run it
Copilot	Multi-turn chat over one dataset
Root cause	Why a metric changed
What-if	Scenario arithmetic
Scheduled	Recurring questions and alerts
Dashboards	KPIs, charts, narrative, Chart Studio, inspect, chat, sharing, reruns
Reports	Downloadable summaries
Audit trail	Code, data lineage and verifications behind every number
Data quality	Checks, drift, incidents
Semantic layer	Approved metric definitions
Evaluation	Agent regression-suite results
Workspaces	Communities, teams, invites
Team overview	Timeline of the team's analyses
Settings	Account, team and backend configuration
Shared (no sidebar)	Public read-only dashboard opened from a share link